

MaixCAM MaixPy 识别 Apritag 标签

2024-04-03

▼ 更新历史

日期	版本	作者	更新内容
2024-04-03	1.0.0	lxowalle	初版文档

阅读本文前，确保已经知晓如何开发MaixCAM，详情请阅读 [快速开始](#)

一、简介

本文介绍如何使用MaixPy来识别Apritag标签

二、使用 MaixPy 识别 Apritag 标签

MaixPy的 `maix.image.Image` 中提供了 `find_apriltags` 方法，可以识别apritag标签。

2.1. 如何识别Apritag标签

一个简单的示例，实现识别apritag标签并画框

```
1 from maix import image, camera,
2 display
3
```

```

4  cam = camera.Camera()
5  disp = display.Display()
6
7  families =
8  image.ApriltagFamilies.TAG36H11
9  x_scale = cam.width() / 160
10 y_scale = cam.height() / 120
11
12 while 1:
13     img = cam.read()
14
15     new_img = img.resize(160, 120)
16     apriltags =
17 new_img.find_apriltags(families =
18 families)
19     for a in apriltags:
20         corners = a.corners()
21
22         for i in range(4):
23             corners[i][0] =
24 int(corners[i][0] * x_scale)
25             corners[i][1] =
26 int(corners[i][1] * y_scale)
27             x = int(a.x() * x_scale)
28             y = int(a.y() * y_scale)
29             w = int(a.w() * x_scale)
30             h = int(a.h() * y_scale)
31
32             for i in range(4):
33                 img.draw_line(corners[i]
34 [0], corners[i][1], corners[(i + 1)
35 % 4][0], corners[(i + 1) % 4][1],
36 image.COLOR_RED)
37                 img.draw_string(x + w, y,
38 "id: " + str(a.id()),
39 image.COLOR_RED)
40                 img.draw_string(x + w, y +
41 15, "family: " + str(a.family()),
42 image.COLOR_RED)
43
44         disp.show(img)

```

步骤:

1. 导入image、camera、display模块

```
1 | from maix import image, camera,  
   | display
```

2. 初始化摄像头和显示

```
1 | cam = camera.Camera()  
2 | disp = display.Display()
```

3. 从摄像头获取图片并显示

```
1 | while 1:  
2 |     img = cam.read()  
3 |     disp.show(img)
```

4. 调用 `find_apriltags` 方法识别摄像头图片中的 apriltag标签

```
1 | new_img = img.resize(160, 120)  
2 | apriltags =  
   | new_img.find_apriltags(families  
   | = families)
```

- `img` 是通过 `cam.read()` 读取到的摄像头图像
- `img.resize(160, 120)` 是用来将图像缩放得更小，用更小的图像来让算法计算得更快
- `new_img.find_apriltags(families = families)` 用来寻找apriltag标签，并将查询结果保存到 `apriltags`，以供后续处理。其中 `families` 用来选择apriltag族，默认为 `image.ApriltagFamilies.TAG36H11`

5. 处理识别标签的结果并显示到屏幕上

```

1  for a in apriltags:
2      # 获取位置信息（并映射坐标到原
3      图）
4      x = int(a.x() * x_scale)
5      y = int(a.y() * y_scale)
6      w = int(a.w() * x_scale)
7      corners = a.corners()
8      for i in range(4):
9          corners[i][0] =
10     int(corners[i][0] * x_scale)
11          corners[i][1] =
12     int(corners[i][1] * y_scale)
13
14     # 显示
15     for i in range(4):
16
17         img.draw_line(corners[i][0],
18                       corners[i][1], corners[(i + 1)
19 % 4][0], corners[(i + 1) % 4]
20 [1], image.COLOR_RED)
21         img.draw_string(x + w,
22 y, "id: " + str(a.id()),
23 image.COLOR_RED)
24         img.draw_string(x + w,
25 y + 15, "family: " +
26 str(a.family()),
27 image.COLOR_RED)
28         img.draw_string(x + w,
29 y + 30, "rotation : " + str(180
30 * a.rotation() // 3.1415),
31 image.COLOR_RED)

```

- 遍历 `apriltags` 的成员，`apriltags` 是通过 `img.find_apriltags()` 扫描 `apriltag` 标签的结果，如果找不到标签则 `apriltags` 的成员为空
- `x_scale` 和 `y_scale` 用来映射坐标，由于 `new_img` 是缩放后的图像，计算 `apriltag` 的坐标时需要经过映射后才能正常的画在原图 `img` 上

- `a.corners()` 用来获取已扫描到的标签的四个顶点坐标, `img.draw_line()` 利用这四个顶点坐标画出标签的形状
- `img.draw_string` 用来显示标签的内容, 其中 `a.x()` 和 `a.y()` 用来获取标签左上角坐标x和坐标y, `a.id()` 用来获取标签的id, `a.family()` 用来获取标签族类型, `a.rotation()` 用来获取标签的旋转角度。

2.2. 常用参数说明

列举常用参数说明, 如果没有找到可以实现应用的参数, 则需要考虑是否使用其他算法实现, 或者基于目前算法的结果扩展所需的功能

参数	说明	
roi	设置算法计算的矩形区域, roi=[x, y, w, h], x, y表示矩形区域左上角坐标, w, h表示矩形区域的宽度和高度, 默认为整张图片	计算坐标 宽和高为 <code>img.fin</code> <code>100, 100</code>
families	apriltag标签家族类型	扫描TAG <code>img.fin</code> <code>image.Ap</code>

本文介绍常用方法, 更多 API 请看 API 文档的 [image](#) 部分。

2.3. 测距1：物体垂直与摄像头的距离

这里提供一种使用 $\text{distance} = k / \text{width}$ 的公式来测距, 其中 distance 是摄像头和物体的距离, 单位 mm , k 是一个常量, width 是物体在画面中的宽度, 单位是像素点.

测量方法分两步: 1. 测量常量系数 k ; 2. 通过常量系数和标签宽度来计算物体与摄像头的距离

2.3.1. 前期准备

1. `apriltag` 标签纸
2. 尺子(或其他测距工具)

2.3.2. 测量常量系数 k

- 将 `apriltag` 标签纸固定, 并在距离 `apriltag` 标签 20cm 处固定 `maixcam`
- 使用 `maixcam` 检测 `apriltag` 标签并计算标签的宽度, 参考代码:

```
1  from maix import camera,
2  display
3  import math
4
5  '''
6  x1,y1,x2,y2: apriltag宽度的两点坐标, 一般通过corners()方法获取
7  返回标签的宽度, 单位为像素点
8  '''
9
10 def caculate_width(x1, y1, x2,
11                    y2):
12     return math.sqrt((x2 -
13                      x1)**2 + (y2 - y1)**2)
14
15 cam = camera.Camera(160, 120)
16 disp = display.Display()
17
18 while 1:
19     img = cam.read()
```

```

20
21     apriltags =
22 img.find_apriltags()
23     for a in apriltags:
24         corners = a.corners()
25
        # 通过水平方向的两个坐标点
        计算宽度
        width =
        caculate_width(corners[0][0],
        corners[0][1], corners[1][0],
        corners[1][1])
        # 打印apriltag标签的实际
        宽度
        print(f'apriltag width:
        {width}')
        disp.show(img)

```

- 计算常量系数k

```

1  '''
2  width: 当距离为distance时,检测到
3  apriltag标签的宽度
4  distance: 检测apriltag标签时距离
5  apriltag标签的实际距离, 单位mm
6  返回常量系数
7  '''
8  def caculate_k(width,
9  distance):
10     return width * distance

    # 距离200mm时检测到标签宽度为43个像
    素
    k = caculate_k(43, 200)

```

2.3.3. 通过常量系数计算摄像头和物体间的距离

```

1  '''
2  width: apriltag标签的宽度

```

```

3 k: 常量系数
4 返回摄像头与物体的距离,单位mm
5 '''
6 def caculate_distance(width, k):
7     return k / width
8
9 distance = caculate_distance(55,
    8600)

```

2.3.4. 完整的代码参考:

```

1 from maix import camera, display,
2 image
3 import math
4
5 '''
6 x1,y1,x2,y2: apriltag宽度的两点坐标,
7 一般通过corners()方法获取
8 返回标签的宽度,单位为像素点
9 '''
10 def caculate_width(x1, y1, x2, y2):
11     return math.sqrt((x2 - x1)**2 +
12 (y2 - y1)**2)
13
14 '''
15 width: 当距离为distance时,检测到
16 apriltag标签的宽度
17 distance: 检测apriltag标签时距离
18 apriltag标签的实际距离, 单位mm
19 返回常量系数
20 '''
21 def caculate_k(width, distance):
22     return width * distance
23
24 '''
25 width: apriltag标签的宽度
26 k: 常量系数
27 返回摄像头与物体的距离,单位mm
28 '''
29 def caculate_distance(width, k):
30     return k / width
31

```

```

32
33
34 cam = camera.Camera(192, 108)
35 disp = display.Display()
36
37 # 距离200mm时检测到标签宽度为43个像素
38 k = caculate_k(43, 200)
39
40 while 1:
41     img = cam.read()
42
43     apriltags = img.find_apriltags()
44     for a in apriltags:
45         corners = a.corners()
46         for i in range(4):
47             img.draw_line(corners[i]
48 [0], corners[i][1], corners[(i + 1)
49 % 4][0], corners[(i + 1) % 4][1],
50 image.COLOR_GREEN)
51
52         # 通过水平方向的两个坐标点计算
        宽度
        width =
        caculate_width(corners[0][0],
        corners[0][1], corners[1][0],
        corners[1][1])

        # 计算距离
        distance =
        caculate_distance(width, k)

        print(f'apriltag width:
        {width} distance:{distance} mm')

        disp.show(img)

```

上面的方法是通过 `apriltag` 的宽度计算距离, 同样也可以扩展为使用高度来计算距离. 但需要注意该方法是在对距离估测, 实际应用中会有些许误差存在.

2.4. 测距2：利用apriltag标签测量物体到摄像头的距离

通过 `apriltag` 标签测距，可以比较准确的测量标签在空间中的位置，这里我们也利用 `find_apriltag()` 方法返回的参数来计算任意位置的 `apriltag` 标签到摄像头的位置，当然前提是你必须检测到 `apriltag` 标签。

测量方法共两步：1. 计算常量系数k。2.通过 `find_apriltag()` 返回的位置信息计算标签到摄像头的距离

优点：可以在 `apriltag` 标签旋转、与摄像头有偏移的情况测量出标签到摄像头的距离

2.4.1. 前期准备

1. `apriltag` 标签纸
2. 尺子(或其他测距工具)

2.4.2. 测量常量系数

- 将 `apriltag` 标签纸固定,并在距离 `apriltag` 标签 20cm处固定 `maixcam`
- 使用 `maixcam` 检测 `apriltag` 标签并通过 `z_translation` 计算常量系数, 参考代码:

```
1  from maix import camera,  
2  display  
3  
4  '''  
5  z_trans: 当距离为distance时,检测  
6  到apriltag标签z_translation()的值  
7  distance: 检测apriltag标签时距离  
8  apriltag标签的实际距离, 单位mm  
9  返回常量系数
```

```

10     '''
11     def caculate_k(z_trans,
12     distance):
13         return distance / z_trans
14
15     cam = camera.Camera(160, 120)
16     disp = display.Display()
17
18     while 1:
19         img = cam.read()
20
21         apriltags =
img.find_apriltags()
        for a in apriltags:
            k =
caculate_k(a.z_translation(),
200)
            print(f"k:{k}")
            disp.show(img)

```

2.4.3. 测量标签到物体的距离

- 通过 `apriltag` 标签返回的

`x_translation`, `y_translation` 和

`z_translation` 计算标签到摄像头的距离

```

1     '''
2     x_trans: 检测apriltag标签返回的
3     x_translation()的值
4     y_trans: 检测apriltag标签返回的
5     y_translation()的值
6     z_trans: 检测apriltag标签返回的
7     z_translation()的值
8     k: 常量系数
9     返回距离, 单位mm
10    '''
11
12    def calculate_distance(x_trans,
13    y_trans, z_trans, k):
14        return k *
15        math.sqrt(x_trans * x_trans +
16        y_trans * y_trans + z_trans *

```

```
z_trans)
```

2.4.4. 完整的代码参考:

```
1  from maix import camera, display,
2  image
3  import math
4
5  '''
6  z_trans: 当距离为distance时,检测到
7  apriltag标签z_translation()的值
8  distance: 检测apriltag标签时距离
9  apriltag标签的实际距离, 单位mm
10  返回常量系数
11  '''
12  def caculate_k(z_trans, distance):
13      return distance / z_trans
14
15  '''
16  x_trans: 检测apriltag标签返回的
17  x_translation()的值
18  y_trans: 检测apriltag标签返回的
19  y_translation()的值
20  z_trans: 检测apriltag标签返回的
21  z_translation()的值
22  k: 常量系数
23  返回距离, 单位mm
24  '''
25  def calculate_distance(x_trans,
26  y_trans, z_trans, k):
27      return abs(k * math.sqrt(x_trans
28  * x_trans + y_trans * y_trans +
29  z_trans * z_trans))
30
31  cam = camera.Camera(160, 120)
32  disp = display.Display()
33
34  # 距离200mm时检测到apriltag标签返回的
35  z_translation()为-9.7
36  k = caculate_k(-9.7, 200)
37
38
```

```

39 while 1:
40     img = cam.read()
41
42     apriltags = img.find_apriltags()
43     for a in apriltags:
44         corners = a.corners()
         for i in range(4):
             img.draw_line(corners[i]
[0], corners[i][1], corners[(i + 1)
% 4][0], corners[(i + 1) % 4][1],
image.COLOR_GREEN)

         # 计算距离
         x_trans = a.x_translation()
         y_trans = a.y_translation()
         z_trans = a.z_translation()
         distance =
calculate_distance(x_trans, y_trans,
z_trans, k)
         print(f'apriltag k:{k}
distance:{distance} mm')

disp.show(img)

```

这段代码使用 **MaixCAM** 不断读取摄像头图像，检测 **apriltag**，并通过常量系数以及标签在三个轴向的位移计算标签到摄像头的距离。计算出的距离以毫米为单位打印出来，需要注意该方法仍是在对距离估测，实际应用中会有些许误差存在。